

Atacando um Enxame com um Bando de Mentirosos: vulnerabilidades em BitTorrent

Marlom A. Konrath¹, Marinho P. Barcellos¹, Juliano F. Silva¹
Luciano P. Gaspar², Rafael Dreher¹

¹ PIPCA – Programa de Pós-Graduação em Computação Aplicada
Universidade do Vale do Rio dos Sinos (UNISINOS) – São Leopoldo, RS – Brasil

²Instituto de Informática – Universidade Federal do Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

Abstract. *BitTorrent is a file sharing technology associated with a significant fraction of the Internet traffic. Considering its importance and large scale adoption, the BitTorrent architecture should be robust and resistant to malicious peers. This paper addresses vulnerabilities in BitTorrent, focusing on two Sybil-based attacks (false virtual identities). It has three contributions: it identifies two attacks and evaluates their potential impact; it increases the understanding of the BitTorrent protocol, contrasting the protocol specification and popular implementations; it brings a discrete event simulation which maps the behaviour of the protocol and has been validated against an experimental evaluation performed in a controlled environment.*

Resumo. *BitTorrent é uma tecnologia de compartilhamento de arquivos responsável por fração significativa do tráfego na Internet. Considerando sua importância e adoção em larga escala, a arquitetura BitTorrent precisa ser robusta e resistente a pares maliciosos. Este trabalho trata de vulnerabilidades em BitTorrent, com ênfase em dois ataques baseados em Sybils (identidades virtuais falsas). São três as contribuições: identificar os ataques e avaliar seu impacto potencial; aumentar o entendimento sobre o BitTorrent, documentando-se o comportamento de implementações populares; e a criação de uma simulação discreta que mapeia o comportamento do protocolo, validada comparando-se com uma avaliação experimental em ambiente controlado.*

1. Introdução

O paradigma Peer-to-Peer (P2P), embora seja um conceito bastante antigo e relacionado ao surgimento da Internet [Sadok et al. 2005], tem adquirido grande visibilidade nos últimos anos. Isso se deve em grande parte à popularidade das redes P2P de compartilhamento de arquivos. Prova disso é que aproximadamente 80% do tráfego em backbones IP em 2005 foi associado a P2P [Barbera et al. 2005]. Existem as diferentes tecnologias de compartilhamento de arquivos, tais como Gnutella, KaZaa e BitTorrent. Esta última, foco do presente trabalho, é senão a mais, uma das mais populares atualmente. Por exemplo, BitTorrent foi responsável em 2004 por 53% de todo o tráfego P2P [Barbera et al. 2005].

Embora BitTorrent¹ possa ser usado para disseminação eficiente de qualquer conteúdo, tem sido usada predominantemente para distribuição ilegal de conteúdo protegido por *copyright*. Entretanto, nos últimos meses grandes empresas, como Warner Brothers, 20th Century Fox² e BBC³ indicaram que passarão a distribuir conteúdo através dessa tecnologia.

¹Neste artigo, o termo *BitTorrent* é usado para designar o protocolo, a arquitetura e a tecnologia BitTorrent, intercambiavelmente e dependendo do contexto.

²<http://www.bittorrent.com/2006-11-29-Digital-Download-Deals.html>

³<http://news.bbc.co.uk/2/hi/technology/6194929.stm>

Considerando sua importância e adoção em larga escala no contexto das redes P2P, ataques a BitTorrent possuem grande impacto potencial. Como é comum, BitTorrent não foi projetado considerando requisitos de segurança e possui, portanto, vulnerabilidades passíveis de serem exploradas.

A contribuição principal deste trabalho é identificar possibilidades de ataques ao protocolo BitTorrent e avaliar seu impacto potencial no andamento de enxames. Ao entender vulnerabilidades e seus impactos, pesquisadores podem propor contramedidas ou modificações na arquitetura de maneira a aperfeiçoar a mesma. Trabalhos anteriores se concentram na modelagem do BitTorrent, particularmente na avaliação do mecanismo de incentivo e/ou como o mesmo pode ser burlado para ganho pessoal. Ao que se sabe, o presente artigo representa a primeira contribuição em termos de vulnerabilidades do BitTorrent. A segunda contribuição é aumentar o entendimento sobre BitTorrent, com base na especificação do protocolo e traços gerados por suas implementações. A terceira é a criação de uma simulação discreta que mapeia o comportamento do protocolo, e que é validada comparando-se com uma avaliação experimental em ambiente controlado.

O restante do artigo está estruturado como segue. A Seção 2 discute os trabalhos relacionados, enquanto a Seção 3 aborda os componentes e o funcionamento da arquitetura BitTorrent. Na Seção 4, são identificados ataques visando prejudicar o espalhamento de peças. A Seção 5 descreve a simulação do BitTorrent, sendo esta simulação usada para avaliar o impacto de ataques, conforme apresentado na Seção 6. Comentários finais e trabalhos futuros são indicados na Seção 7.

2. Trabalhos Relacionados

Diversos trabalhos na literatura demonstram primeiro que o mecanismo de incentivo utilizado pelo BitTorrent nem sempre garante justiça no compartilhamento, para então propor novos mecanismos de interação. Em [Purandare and Guha 2006], é proposto e simulado um modelo para BitTorrent que agrupa usuários em categorias e os convence a cooperar. Em [Bharambe et al. 2005a], autores estudam por simulação a política de *tit-for-tat* e concluem que a mesma não é efetiva para se evitar injustiça, e propõem uma modificação no tracker para aliviar o problema. Já em [Bharambe et al. 2005b], é demonstrada a robustez do BitTorrent, analisando-se os componentes básicos via simulação, para diferentes cargas de trabalho, em termos de utilização de enlace upstream, tempo de download e justiça; os autores então propõem técnicas para lidar com as situações específicas em que BitTorrent não apresenta comportamento eficiente. Em [Guo et al. 2005], os autores focam na participação de pares em múltiplos enxames, e então estudam BitTorrent considerando fatores realísticos, identificando limitações que incluem indisponibilidade, desempenho ruim e instável por parte de pares, e injustiça. Em [Jun and Ahamad 2005], os autores mostram que o mecanismo de incentivo não é totalmente eficaz em desestimular pares-carona, propondo sua modificação. Em contraste, em [Legout et al. 2006] defende-se que os mecanismos de *Local Rarest First* (LRF) e *tit-for-tat* são suficientemente bons e que sua substituição não se justificaria, pois outros modelos aumentariam a complexidade da solução e não trariam melhora significativa.

Diversos trabalhos [Qiu and Srikant 2004, Yang and de Veciana 2004, Veciana and Yant 2003] estudam a capacidade de serviço do sistema BitTorrent com população transeunte. São apresentados modelos matemáticos baseados em cadeias de Markov e na teoria dos fluídos como forma de modelar o comportamento da arquitetura.

O trabalho mais fortemente relacionado é [Liogkas et al. 2006], que apresenta três técnicas que um par poderia empregar no sentido de aproveitar mais da rede do que contribuir para a mesma. Tal como aquele, o presente artigo explora técnicas em que um par não respeita o protocolo e “mente” (sobre a disponibilidade de peças) ou se recusa a cooperar. Diferentemente dele, o anúncio falso de peças é usado para prejudicar, ou mesmo evitar, o espalhamento de conteúdo. Além disso, são explorados cenários em que a quantidade de atacantes é muito

superior a de honestos, causando um “isolamento” dos pares honestos. Ao que se sabe, até o presente momento não há estudo que ofereça um estudo sobre vulnerabilidades da arquitetura e o impacto das mesmas.

3. Arquitetura BitTorrent

Esta seção fornece uma descrição geral do funcionamento do protocolo (Subseção 3.1). A seguir, aborda duas políticas importantes no BitTorrent, que serão alvo de ataques: a escolha da próxima peça a ser obtida por um par (Subseção 3.2), e a política para conexão com outros pares (Subseção 3.3).

3.1. Arquitetura e Protocolos

A seguir, são descritos os principais elementos da arquitetura BitTorrent. O “conteúdo” (um conjunto de arquivos) a ser disseminado em uma rede BitTorrent é descrito através de um arquivo de metadados e de terminação “.torrent”. Tal arquivo é gerado pelo usuário que publica o conteúdo, e contém informações como o nome e tamanho dos arquivos, *hashes* para os dados, e um ou mais endereços IP de *trackers*. Um tracker é um elemento centralizado que coordena um “enxame” (*swarm*) e auxilia pares a encontrar outros pares no mesmo enxame. Um enxame é um conjunto de pares que faz upload ou download de um mesmo conteúdo e conecta-se periodicamente com um tracker. Pares podem ser separados entre “semeadores” (*seeders*), que são pares que possuem uma cópia completa dos dados, e “sugadores” (*leechers*), que são aqueles que ainda não completaram seus downloads. O conteúdo publicado e descrito no .torrent é organizado em “peças”, e peças subdivididas em “blocos”. Blocos são a unidade mínima de troca de dados entre pares. Pares estabelecem conexões TCP entre si e trocam mapas de bits (*bitfields*) contendo informações sobre disponibilidade de peças. Um par solicita a um ou mais pares o download de blocos de uma determinada peça em que está interessado. Solicitações são atendidas através de um mecanismo de incentivo baseado em reciprocidade, conforme explicado a seguir.

O mecanismo de incentivo é utilizado para aumentar a cooperação entre os nodos. Baseia-se em o par avaliar, a cada rodada, os pares que mais forneceram conteúdo a ele e retribuir, na rodada seguinte, atendendo suas requisições. Tipicamente, cada rodada dura aproximadamente 10 segundos; o conteúdo fornecido nos últimos 30 segundos é considerado no cálculo e os 3 pares remotos que mais contribuíram são escolhidos para receber upload a partir do par local. Os pares remotos para os quais um par local está realizando upload são marcados como “não sufocados” (*unchoked*). Os demais tem o envio de conteúdo “sufocado” (*choked*) pelo cliente. O cliente sempre sabe quais pares o estão sufocando e quais não, pois cada vez que um cliente muda o estado de um par, uma mensagem é enviada ao par informando-o. Além de três pares marcados como não sufocados em função de sua contribuição, um quarto par, escolhido de forma aleatória entre os conectados, é eleito para receber conteúdo. A escolha de um nodo desconsiderando sua contribuição é chamada de *optimistic unchoke* e é a forma utilizada pelo protocolo para tentar encontrar pares como melhores taxas de download e upload que as atuais.

Na ausência de uma descrição formal do protocolo, foram criados diagramas de estado e de tempo, através da análise da descrição do protocolo [BitTorrentProtocolSpecification 2006] e de traços de implementações populares do protocolo capturadas em ambiente controlado. A Figura 1 apresenta o diagrama de estados, de forma global e com ênfase na troca de peças. Os elementos principais são descritos a seguir.

Um par inicia no estado START, conforme representado na Figura 1. Ao ser iniciado, o par solicita uma conexão com o tracker e passa ao estado WAIT_TRKCONNECT. No estado WAIT_TRKCONNECT, se a conexão é recusada, o par configura um temporizador para uma tentativa e volta ao estado START; se há um timeout, ocorre uma nova tentativa. Caso a conexão tenha sido estabelecida com sucesso, o par envia um **Get** ao tracker e passa ao estado WAIT_PEERLIST.

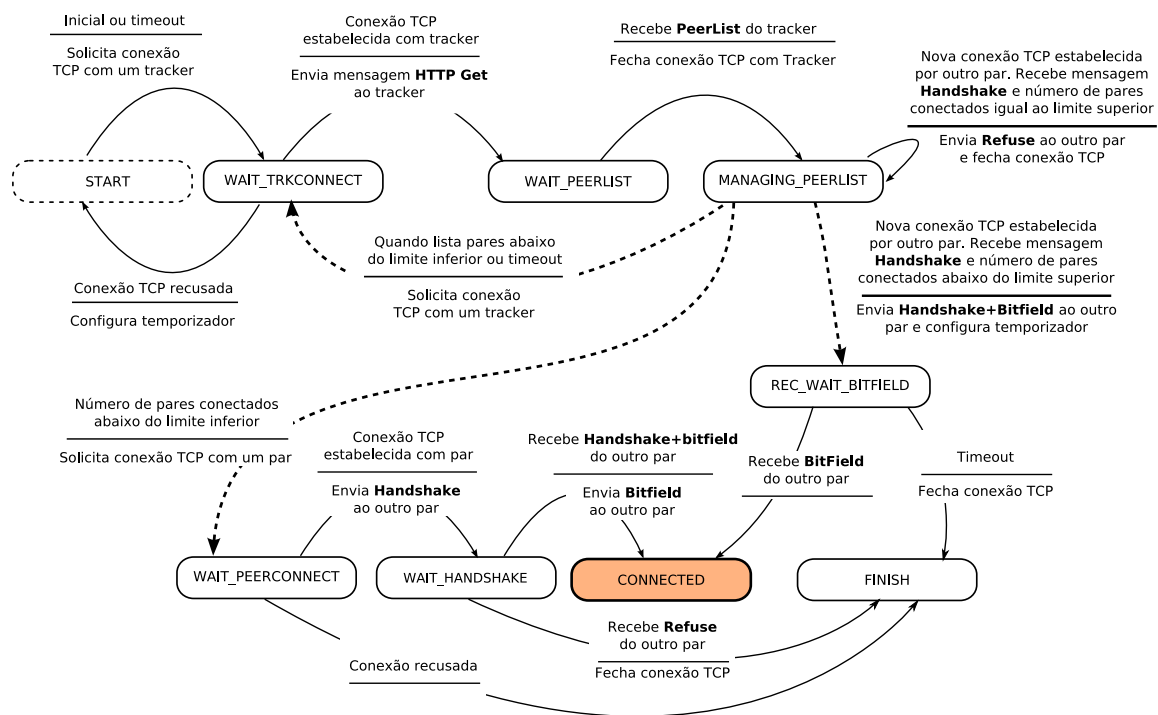


Figura 1. Diagrama de estados do BitTorrent

No estado `WAIT_PEERLIST`, quando o par recebe a mensagem **PeerList**, ele fecha a conexão com o tracker, inicia um temporizador para que o tracker seja re-consultado e passa ao estado `MANAGING_PEERLIST`. Se o arquivo .torrent possuir mais de um tracker, estas iterações ocorrerão de forma independente, uma para cada tracker.

No estado `MANAGING_PEERLIST`, há quatro transições possíveis: primeiro, se a lista de pares fica muito pequena ou chega o momento de reconsultar o tracker (timeout), um novo fluxo de execução⁴ é criado no estado `WAIT_TRKCONNECT`, de forma a conectar o par com o tracker e obter mais pares. Segundo, considerando que o par, ao conectar com o tracker, informou seu identificador e pode ser eleito nas próximas listas que o tracker enviar a outros pares, múltiplas conexões podem ser solicitadas por pares remotos; para cada uma, o par local verifica o limite do número de conexões. Caso dentro do limite, envia mensagem **Handshake+Bitfield**, configura temporizador e cria thread no estado `REC_AWAIT_BITFIELD` para gerenciar a comunicação com aquele par. No entanto, se o número de pares conectados tiver excedido o limite, uma mensagem **Refuse** é retornada ao par remoto e a conexão com o mesmo fechada. A quarta possibilidade é a criação de uma thread no estado `WAIT_PEERCONNECT`, que irá providenciar a conexão com um dos pares da lista. Esta última transição ocorre uma vez para cada um dos pares escolhidos, com base na lista recebida em **PeerList**, até que o número de pares conectados atinja o limite ou que não existam mais pares na lista.

No estado `REC_AWAIT_BITFIELD`, a thread aguarda a chegada do **Bitfield**, quando passa ao estado `CONNECTED`, refletindo a conexão operante com um par remoto. Se a resposta não chegar, ocorre um timeout, levando ao fechamento da conexão e o encerramento da thread. No estado `WAIT_PEERCONNECT`, a thread aguarda a conexão com o par remoto associado à mesma. Se o estabelecimento de conexão falha, há uma transição para `FINISH` e a thread termina. Se completada com sucesso, envia uma mensagem **Handshake** e passa ao estado `WAIT_HANDSHAKE`. Neste estado, a thread pode receber do par remoto uma mensagem **Refuse** ou **Handshake+Bitfield**. No primeiro caso, a conexão TCP com o par remoto é fechada e a thread passa ao estado `FINISH`,

⁴uma nova thread, que é representada por uma seta pontilhada

sendo eliminada. No segundo, responde ao par remoto enviando seu mapa de bits em **Bitfield** e passando ao estado **CONNECTED**.

O estado **CONNECTED**, marcado no diagrama, concerne às variações de estado que uma thread pode sofrer em função de sua relação com o par remoto pelo qual está responsável de interagir. Essa relação é dirigida por dois diagramas de estado, um para download e outro para upload. O único vínculo entre os mesmos é o processo de escolha de pares, segundo o mecanismo de incentivo, para os quais realizar upload; tal escolha é baseada em reciprocidade em termos de downloads desses pares. As Figuras 2 e 3 representam os diagramas.

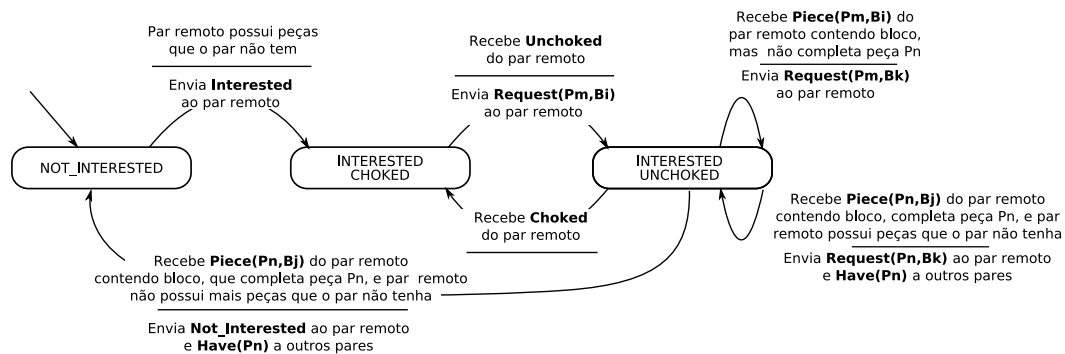


Figura 2. Diagrama de estados de thread em relação a downloads de par remoto

Para efeito de download, uma thread em **CONNECTED** está, inicialmente, no estado **NOT_INTERESTED**. Examinando o mapa de bits recebido do par remoto, a thread pode verificar que o par remoto possui uma ou mais peças que o par local não possui. Nesse caso, a thread passa ao estado **INTERESTED_CHOKED**, e envia uma mensagem **Interested** indicando que o par remoto é considerado “interessante”; entretanto, a thread está sendo sufocada, e assim não pode ainda solicitar peças ao par remoto. No estado **INTERESTED_CHOKED**, a thread aguarda uma mensagem **Unchoked**; quando isso acontece, é um indicativo do par remoto que o par local está autorizado a solicitar blocos. A thread então envia uma mensagem **Request(Pm,Bi)** e passa ao estado **INTERESTED_UNCHOKED**. Nesse estado, quatro transições são possíveis. Primeiro, um bloco de uma peça é recebido, porém não completa a peça; uma mensagem **Request(Pm,Bk)** é enviada para requisitar um outro bloco dessa mesma peça. Segundo, um bloco de uma peça é recebido, completando-a; neste caso, há ainda duas possibilidades, dependendo se o par remoto possui outras peças que o local não tem. Caso sim, uma mensagem **Request(Pm,Bk)** é enviada solicitando um bloco de outra peça, e a thread permanece no mesmo estado. Caso não, envia mensagem **Not_Interested** ao par remoto e passa ao estado **NOT_INTERESTED**. Em ambos os casos, uma mensagem **Have(Pn)** é enviada aos demais pares conectados, anunciando a disponibilidade da peça.

Para efeito de upload, uma thread em **CONNECTED** inicia em **NON_INTERESTING**, indicando que este par não é interessante ao par remoto. Quando o par recebe uma mensagem **Interested**, ele passa ao estado **INTERESTING_CHOKING**: o par local tem peças que o remoto deseja, mas está sufocando o mesmo. Nesse estado, é possível que o par remoto declare, através de uma mensagem **Not_Interested**, não estar mais interessado (por exemplo, porque obteve todas as peças que este par possui).

Quando o par remoto entra na lista dos escolhidos para upload, a thread envia mensagem **Unchoked** ao remoto e passa ao estado **UNCHOKING**. Enquanto nesse estado, o par recebe múltiplas requisições de blocos em mensagens **Request(Pm,Bj)**, enviando a resposta em **Piece(Pm,Bj)**. No momento em que par remoto sai da lista de escolhidos para upload, a thread envia uma mensagem **Choked** e volta ao estado **INTERESTING_CHOKING**.

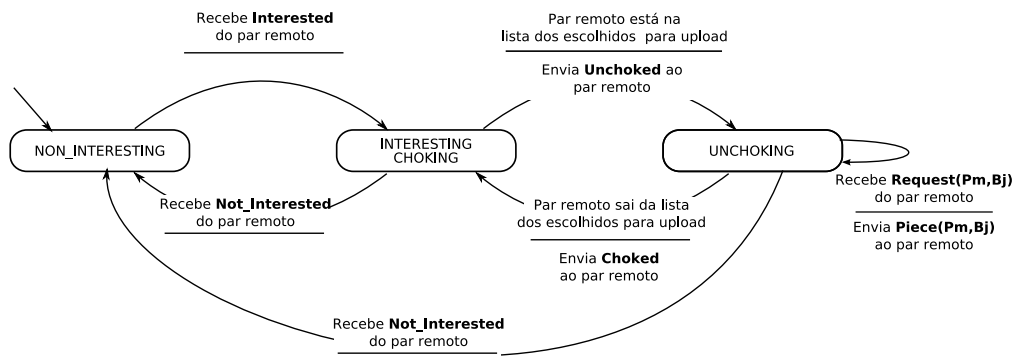


Figura 3. Diagrama de estados de thread em relação a uploads para par remoto

3.2. Política de Seleção de Peças

O sucesso de um par no seu download, bem como do enxame como um todo, é influenciado pela política de seleção de peças aplicada pelo par. A seleção no BitTorrent segue a política da “Peça Local Mais Rara Primeiro” (*Local Rarest First*). Ela se baseia no conceito de escolher primeiro as peças que estão menos replicadas entre pares remotos com os quais o par está conectado; caso exista mais de uma peça com grau mínimo de replicação, a escolha é aleatória.

Existem três exceções para essa regra: quando o nodo está iniciando o download (situação em que possui menos de 4 peças), finalizando-o (já requisitou todos os blocos que faltam) ou atuando como semeador (possui todas as peças). Há três aspectos positivos nessa abordagem: mecanismo faz com que a robustez da rede aumente, já que todas as peças terão em média a mesma quantidade de réplicas entre os pares; um par que entra na rede e não possua peças, ao escolher as mais raras, aumentará sua chance de possuir conteúdo que outros pares com mais peças não possuem; no fim do processo de download, a política garante que as peças que faltam estarão distribuídas de forma mais ou menos uniforme.

Segundo [Legout et al. 2006], o uso da política LRF na escolha evita os problemas da “peça rara” e da “última peça”. O primeiro problema ocorre quando uma ou mais peças tornam-se muito pouco comuns entre os pares. Já o problema da “última peça” ocorre quando um par está no final do processo de download e precisa de uma peça que está pouco distribuída para completar a obtenção do conteúdo. Em ambos os casos, além da grande probabilidade do download demorar mais, pois a peça estará menos disponível, há também uma maior chance de parte do conteúdo tornar-se indisponível. Por outro lado, a escolha da próxima peça baseia-se na execução correta do protocolo por parte dos pares.

3.3. Política para Conexões com outros Pares

Pares obtêm os endereços IP de outros pares em um enxame através do tracker. O tracker é, literalmente, um elemento central para a arquitetura BitTorrent. Pares que participam de um enxame periodicamente se conectam ao tracker e registram assim sua presença no enxame. Na conexão do par com o tracker, é informado o número de pares que o par local deseja receber (**numwant**), que é por padrão, 50.

Além da lista de pares, o tracker retorna uma série de informações, incluindo **interval**, o tempo em segundos que o par deve esperar antes de realizar outra solicitação. Esse valor também é utilizado para cálculo do tempo que o tracker manterá o IP do cliente na lista de pares ativos. Se o tempo exceder este limite mais certa margem, o tracker considera que o par abandonou o enxame sem informar o mesmo. O tamanho da lista de pares mantida por um tracker é o tamanho de um enxame: varia tipicamente de dois ou três pares até mais de 10.000. Quando solicitado, o tracker monta aleatoriamente uma lista com até 50 pares (ou o valor solicitado em **numwant**) e envia-a ao par solicitante. Quanto maior a lista global no tracker, maior será a diversidade entre as listas fornecidas pelo tracker; quando a lista global do

tracker contém menos que a quantidade de pares a ser enviada não há sorteio – basicamente a mesma lista será fornecida para cada novo par.

Há diversas situações em que um par pode obter a identidade de mais pares. Primeiro, quando um par entra no enxame, ele envia uma mensagem **Get** ao tracker, que responde com o **PeerList**. Segundo, quando a lista decresce abaixo de um limite inferior de 20 pares, o par envia novo **Get** ao tracker. Terceiro, quando o par é contatado por outro par no enxame, sendo que o par remoto não constava da lista do par local. O número máximo de conexões que um par pode manter é normalmente limitada a 55. Tipicamente, o par inicia até 30 conexões com outros pares, sendo as restantes deixadas para conexões iniciadas por pares remotos.

4. Estratégias de Subversão/Ataque

Esta seção trata de ataques a enxames BitTorrent. Primeiro discute-se o ataque *Sybil* em BitTorrent (Subseção 4.1), e então como ele pode servir de base para gerar dois ataques a enxames: de mentira de peças (Subseção 4.2) e de isolamento de pares corretos (Subseção 4.3).

4.1. Sybil

O ataque Sybil [Douceur 2002] em redes P2P consiste em um mesmo par apresentar-se com múltiplas identidades virtuais, usualmente no intuito de burlar sistemas baseados em reputação. Com esse ataque, um par malicioso pode comprometer um sistema P2P ao conseguir o controle de uma fração substancial do mesmo. Os autores sugerem o uso de autoridades certificadoras de identidade como a melhor forma de proteção contra este tipo de ataque.

No BitTorrent, identidades são geradas de forma pseudo-aleatória. Um atacante, portanto, pode valer-se desta vulnerabilidade e obter várias identidades. Pares podem banir um par que se recusa a cooperar, mas nem sempre é fácil identificar quando um par não está cooperando ou quando enfrenta problemas de conectividade, por exemplo.

Assumindo então que um atacante possa executar múltiplos pares compartilhando um mesmo recurso físico (máquina e rede), a questão passa a ser determinar quais ataques seriam mais efetivos em BitTorrent.

4.2. Mentindo a posse de peças

Conforme discutido na Seção 3, pares utilizam dois tipos de mensagens para informar a pares remotos sobre a posse de peças. Logo após o estabelecimento da conexão TCP, pares trocam mensagens **Bitfield** contendo um mapa das peças que cada um possui. Após isso, a cada nova peça obtida, o par envia mensagens **Have** notificando os pares conectados para que atualizem o mapa enviado inicialmente.

Normalmente, os pares buscam uma homogeneidade na quantidade de cópias de cada peça. O ataque Sybil com mentira busca não permitir esta homogeneidade. Um par malicioso pode não executar corretamente o protocolo e anunciar uma peça que não possui, objetivando interferir na escolha da próxima peça pelos outros pares. Ou seja, ao anunciar uma peça, o par malicioso está artificialmente aumentando o nível de replicação da mesma e portanto induzindo pares corretos a fazer download de outras peças primeiro. Em outras palavras, o ataque consiste em fazer com que peças se tornem raras ao ponto de desaparecer do sistema.

Como pares maliciosos não desejam entregar a peça, três estratégias são possíveis: (a) manter sempre outros pares como sufocados; (b) não responder caso um par solicite a peça (ou conjunto de peças) sendo anunciada falsamente, ou (c) enviar lentamente blocos corrompidos, mas não a peça inteira. No primeiro caso, o par malicioso nunca escolherá outros pares para fornecer download mas, assim como quaisquer pares corretos conectados, eles poderão ser selecionados para receber conteúdo. No caso de não responder a requisições, o dano é maior, porque pares maliciosos terão informado pares remotos que estes podem solicitar download, mas o par malicioso jamais responderá às solicitações, levando a timeouts no par correto. No terceiro

caso, o dano é ainda maior, porque além do download ser atrasado, os blocos corrompidos farão com que largura de banda do par correto seja ocupada; no entanto, neste caso, o malicioso precisa arcar com mínima, porém suficiente, largura de banda para garantir o download nos pares atacados.

Além da escolha de estratégia acima, diversos outros aspectos influenciam na eficácia de um ataque, incluindo: tamanho do conteúdo sendo compartilhado, em número de peças; número de peças sendo mentidas por pares maliciosos; tamanho do enxame, em pares, em um determinado momento, e a quantidade de semeadores; grau de disseminação das peças quando do início do ataque; e número de pares maliciosos, e se os mesmos estão atuando em conluio. Um dos objetivos deste trabalho é definir o impacto dos principais fatores em um ataque, conforme será apresentado na Seção 6.

Intuitivamente, mentir a posse de um pequeno conjunto de peças ao invés de apenas uma aumenta a chance de uma delas constituir um problema. No entanto, existe uma relação entre o tamanho desse conjunto e a efetividade do ataque; no extremo, mentir todas as peças seria inefetivo no sentido de tornar uma peça mais rara. O impacto do número de peças mentidas em ataques é avaliado na Seção 6.

O impacto de ataques a um enxame é, pelo menos intuitivamente, muito mais efetivo quando executado por múltiplos pares maliciosos atuando em conluio. No caso específico de tornar uma peça mais rara, a expectativa é que o número de pares maliciosos influencie diretamente no ataque, porque quanto mais pares mentirem ter uma dada peça, menos rara ela *parecerá* se tornar.

4.3. Eclipsando pares corretos

No extremo, se um atacante possui suficientes recursos físicos ou cria um grande número de Sybils, ele pode atacar um enxame fazendo com que um bando de pares maliciosos participe do mesmo. Além disso, um único conjunto de pares maliciosos pode ser usado para atacar múltiplos enxames.

O ataque de Eclipse [Singh et al. 2006] consiste cercar de maliciosos um par correto, de forma que toda sua comunicação possa ser “interceptada”, ou isolando o par ou levando o mesmo a crer que ele está corretamente conectado com o resto da rede.

No contexto deste trabalho, o ataque Eclipse consiste em usar um número suficientemente alto de maliciosos de forma a fazer com que pares corretos estejam conectados predominantemente ou apenas com pares maliciosos. O número máximo de conexões que um par correto mantém é, por padrão, 50. Portanto, para eclipsar pares corretos seria necessário lançar mão de aproximadamente 50 mentirosos para cada par correto sendo atacado. Embora esse seja um número bastante alto (por exemplo 5000 maliciosos para um enxame de 100 pares), considere-se que o tráfego gerado pelo par malicioso está restrito ao plano de controle, pois o mesmo não faz efetivamente download nem upload.

5. Simulação do BitTorrent

A fim de avaliar as proposições na Seção 4, criou-se um **modelo de simulação** do protocolo BitTorrent. Assim como os diagramas de estado, o modelo está baseado em uma combinação da especificação do protocolo e da análise de traços de pacotes de duas implementações populares. Ele incorpora as características do protocolo citadas na Seção 3, com a implementação de trackers, pares e todas as mensagens e estados mostradas nos diagramas daquela seção.

Embora o modelo seja bastante realista, diversas premissas simplificadoras foram adotadas. Primeiro, não há troca efetiva de dados entre pares; portanto, as mensagens de dados carregam apenas a informação de controle necessária. Um par envia mensagens **BLOCK(p,b)** “contendo” um bloco **b** de uma peça **p**. Como não há dados propriamente ditos, não faz sentido usar um mecanismo para detecção de integridade em peças (via hash). Para simular blocos corrompidos, mensagens **BLOCK** incluem um valor booleano que indica se o mesmo é íntegro. Se

todos os blocos de uma peça forem íntegros, então se considera que o cálculo de hash da peça obteve êxito.

Além disso, a simulação adota como premissa o protocolo TCP na camada subjacente, mas não simula o mesmo em detalhe. Na implementação real, um bloco é confiavelmente transferido para outro par, segmento a segmento, através da conexão TCP com o mesmo. Na simulação, uma mensagem **BLOCK** contém um bloco inteiro e é enviada de uma vez só, como se o tamanho do segmento fosse o tamanho do bloco e não houvesse fragmentação na camada de IP. Para simular o recebimento confiável de um bloco de dados na camada de transporte, e permitir o envio do próximo, foi acrescentada a mensagem **TCP_ACK(p,b)**, a ser enviada pelo receptor em resposta a uma mensagem **BLOCK(p,b)**.

Pares podem iniciar como semeadores (com uma cópia completa dos dados) ou sugadores (sem nenhuma peça). Semeadores possuem um tempo configurável de semeadura (cuja contagem inicia no momento que passam de sugadores a semeadores), valor recebido como parâmetro quando da instanciação. Pares possuem um tempo inicial de ativação, valor recebido igualmente no momento da instanciação, e um tempo configurável de timeout (para conexão com o tracker). Os parâmetros restantes para o modelo são o número de peças no conteúdo sendo compartilhado, o número de blocos por peça, e o tamanho de cada bloco em KB.

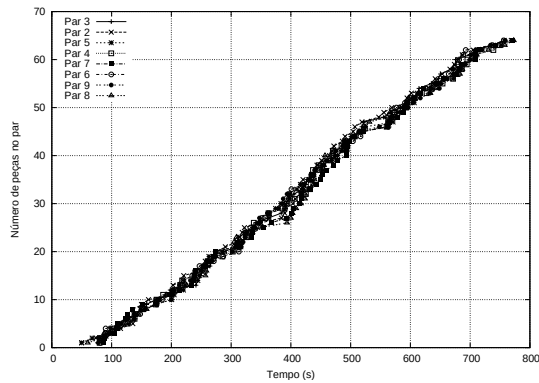
Todos os pares estão conectados por um enlace a uma “nuvem” que representa a rede. A capacidade dos enlaces pode ser individualmente configurada, tanto a partir do par como em direção ao par, em termos de largura de banda e latência fim-a-fim (esta última simulada através de uma distribuição aleatória uniforme). Na comunicação entre dois pares, a capacidade do enlace de saída do par origem ou de chegada no par destino podem ser limitantes (o mesmo se aplica ao tracker).

A **implementação** do modelo de simulação foi realizada em Java, estendendo-se o arcabouço de simulação Simmcast [Simmcast 2006], um simulador discreto para simulação de protocolos distribuídos. A simulação possui aproximadamente 3000 linhas de código Java, mais a lógica para implementação de nodos hospedeiros (HostNode) e roteadores (RouterNode), oferecidas pelo arcabouço do Simmcast. Existem dois tipos de nodos na rede, o tracker (classe TrackerNode) e o par (classe PeerNode). Cada nodo possui pelo menos uma thread contendo a lógica de execução; no caso do tracker ou de pares corretos, nas classes TrackerThread e PeerThread, respectivamente. A “nuvem” é representada por um nodo especial de roteamento (classe P2PRouter), e uma thread (classe P2PRouterThread). A classe CheaterNode representa um par malicioso, que abriga a classe CheaterThread, que por sua vez implementa a lógica do atacante. Além da lógica implementada nas threads do tracker e dos pares semeadores e sugadores, a simulação é definida pelo conjunto de parâmetros acima descritos: número inicial de pares semeadores, número inicial de pares sugadores, número de peças, de blocos por peça e tamanho de blocos e capacidade dos enlaces de chegada e de saída em cada par. Esses parâmetros são informados através de um arquivo texto, não executável, que descreve a simulação.

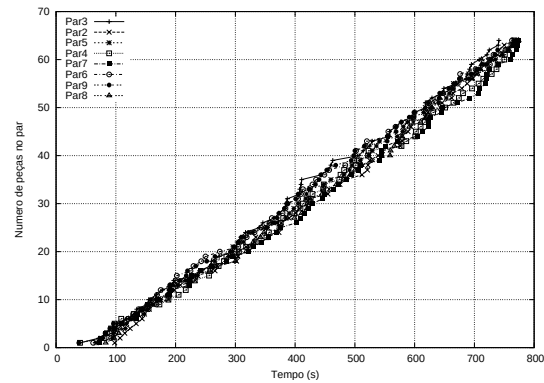
Os resultados de uma execução da simulação podem ser coletados de duas formas: detalhada, no traço de pacotes gerado pelo Simmcast (na classe `simmcast.trace.SimmcastTraceGenerator`), ou mais específica, que são as informações geradas para a saída padrão. O grau de detalhe dessas informações é configurável. O processamento dos dados e consolidação estatística em gráficos são feitos através de scripts em Python, a serem disponibilizados com a simulação.

Considerada uma contribuição secundária deste trabalho, a implementação do modelo de simulação pode ser utilizada para estudar-se o protocolo BitTorrent para outros fins. O código fonte e os scripts de análise estão disponíveis para a comunidade acadêmica, sob licença GPL, no site do Projeto P2P-SeC [P2P-SeC 2006].

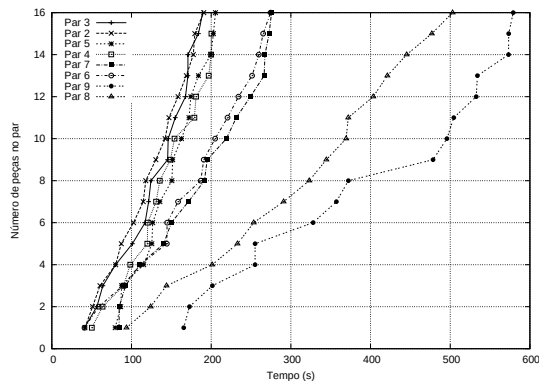
Para permitir a **validação** do modelo de simulação e sua implementação, os resultados de simulação foram comparados a resultados colhidos em experimentos com implementações populares de BitTorrent (Azureus e Mainline). Para tal, um ambiente controlado foi montado



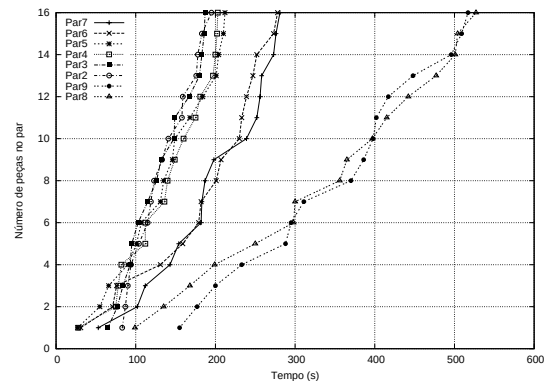
(a) Experimento com 8 pares idênticos



(b) Simulação com 8 pares idênticos



(c) Experimento com 8 pares com taxas diferentes



(d) Simulação com 8 pares com taxas diferentes

Figura 4. Comparação entre avaliação experimental e simulação.

com uma rede local isolada, com um concentrador (*Hub*) interligando 10 máquinas Pentium 4 2.8GHz, 512 MB RAM, HD 40Gb e Windows XP. Para evitar interferências, todos os serviços do Windows foram desabilitados; um programa de captura de pacotes foi executado durante 5 minutos sem ter capturado qualquer pacote na rede. Um limitador de banda em nível de *kernel* foi utilizado para a definição das velocidades de download e upload. O Par1 sempre foi o semeador inicial e o Par10 sempre atuou como tracker do enxame. Como pré-análise, foi realizada uma gama de experimentos variando-se fatores importantes como: número de pares (1, 2, 4 e 8 pares); emulação de largura de banda no enlace (40KB/s, 30KB/s, 20KB/s e 10KB/s, de forma homogênea e heterogênea entre pares); tamanho do arquivo (4MB e 16MB); número de peças (16, 64 e 256 peças) e de blocos (4, 16, 32, 64 blocos). O tráfego foi registrado em dois capturadores de pacotes independentes (*tcpdump* e *wireshark*), executados em máquinas distintas.

Na Figura 4, podem ser observados os resultados obtidos experimentalmente e através de simulação para 8 pares em diferentes configurações. O eixo *x* representa o tempo, em segundos, enquanto o eixo *y* representa o número de peças disponíveis no par. Os pares realizam o download de um arquivo de 16 MB (64 peças de 16 blocos), todos com velocidade igual (pares homogêneos) de 30KB/s. A evolução em relação ao número de peças obtidas ao longo do tempo pode ser observada na Figura 4(a), que se refere ao experimento, e Figura 4(b), à simulação. É possível notar que há grande semelhança entre o caso experimental e a simulação.

No segundo caso – Figuras 4(c) e 4(d) – os pares possuem velocidades diferentes (Pares 1, 2 e 3 com 40KB/s, Pares 4 e 5 com 30KB/s, Pares 6 e 7 com 20KB/s e Pares 8 e 9 com

10KB/s) e realizam o download de um arquivo de 4MB (16 peças de 16 blocos). Novamente, há uma evidente semelhança entre a simulação e o caso experimental. Além disso, os traços gerados pelas simulações foram contrastados com aqueles gerados pelas implementações, no sentido de aumentar a segurança sobre a fidelidade da simulação.

6. Avaliação do Impacto

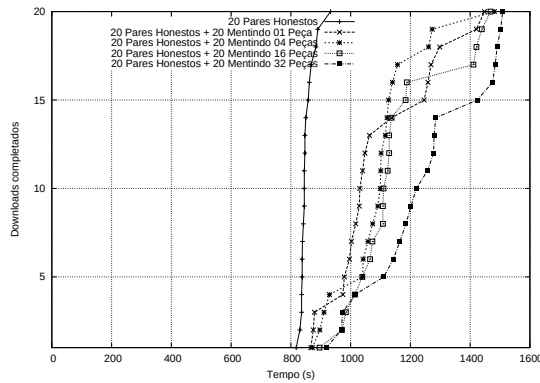
A simulação descrita na seção anterior foi empregada de forma a avaliar cenários em que um enxame BitTorrent é atacado explorando as vulnerabilidades descritas na Seção 4. São avaliados, primeiramente, a mentira de peças, e então os ataques do tipo Eclipse. Para validade estatística, os resultados foram obtidos através de 12 simulações repetidas com sementes aleatórias diferentes.

A simulação segue o modelo descrito na Seção 5. Inicialmente, há apenas um semeador, que é ativado no tempo 0. Sugadores são ativados em tempos distintos, com um atraso uniformemente gerado entre 0 e 10s. Sugadores, quando obtêm o conteúdo completo, se tornam semeadores com probabilidade 0,3 (refletindo os 30% de semeadores reportados em [Andrade et al. 2005]); os que se tornam semeadores permanecem assim até o final da simulação, enquanto os demais terminam. Um par recontacta o tracker pelo menos a cada 30s. Escolheu-se compartilhar um conteúdo de 16MB, dividido em 64 peças de 256KB cada (ou seja, 16 blocos). Tais valores foram escolhidos com base em [Legout et al. 2006]. Os enlaces foram configurados de forma simétrica para download e upload, com cenários homogêneos e heterogêneos. O tracker retorna uma lista de até no máximo 50 pares, um valor frequentemente adotado em implementações. Assume-se que um par não espera eternamente pelo término de um download: as operações que não são completadas dentro de um tempo máximo pré-definido são consideradas falhas.

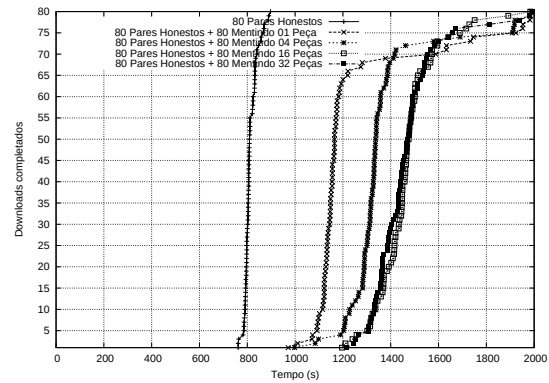
Para avaliar a efetividade de um ataque, foram consideradas duas métricas: atraso médio no término de downloads dos pares participantes do enxame, expresso através do número cumulativo de downloads, e número de falhas em downloads devido à desistência de sugadores (falha do download para um par).

A Figura 5 apresenta dois gráficos que ilustram o impacto do número de peças sendo mentidas por um bando de mentirosos (em igualdade com pares honestos), para enxames com 40 e 160 pares. O gráfico representa o número de downloads completados em função do tempo, para quatro quantidades diferentes de peças sendo mentidas (1, 4, 16 e 32), além do caso sem atacantes. As curvas mostram que a mentira de peças gera uma sobrecarga significativa, atrasando o download dos pares corretos. É possível notar que o número de peças mentidas possui impacto negativo e que são diretamente proporcionais; na Figura 5(a), com 32 peças sendo mentidas, o metade dos pares completa seus downloads no tempo 1200, enquanto os últimos 6 completam após o tempo 1400.

Quanto ao número de atacantes, a Figura 6 apresenta dois gráficos com o impacto do número de pares maliciosos, em enxames com 20 e 80 pares honestos. O gráfico representa o número de downloads completados em função do tempo, para diferentes números de atacantes. As curvas mostram que o impacto do número de atacantes é significativo tanto no cenário com 20 como 80 honestos, causando o atraso dos downloads em geral. Conforme a Figura 6(b), a presença de pelo menos um par mentiroso já provoca um atraso de aproximadamente 400s, ou 50% a mais do que o tempo normal de download. É possível confirmar a intuição que quanto mais atacantes, melhor, pois existe uma tendência geral de que as curvas com maior número de atacantes esteja situada mais à direita no gráfico. Entretanto, é surpreendente que o impacto não seja tão significativo como inicialmente esperado. Exemplificando, na Figura 6(b), 1 mentiroso faz com que 60 downloads tenham sido completados no tempo 1200s, enquanto isso ocorre no tempo 815s quando não há ataque; com 80 mentirosos, os primeiros 60 downloads são completados até o tempo 1500s (300s a mais).

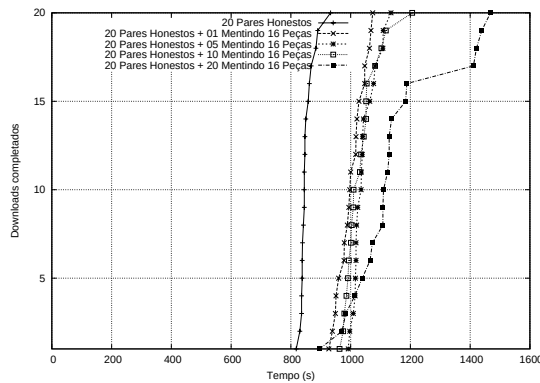


(a) Simulação com 40 pares, metade honestos

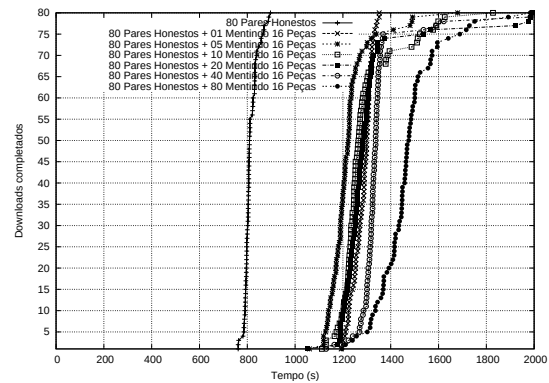


(b) Simulação com 160 pares, metade honestos

Figura 5. Impacto da quantidade de peças mentidas nos downloads, com o mesmo número de pares honestos e maliciosos.



(a) Simulação com 20 pares honestos



(b) Simulação com 80 pares honestos

Figura 6. Impacto da quantidade de mentirosos nos downloads.

No experimento a seguir, a Figura 7 apresenta o percentual de downloads falhos em função do número de atacantes (variando-se entre 100 e 1000 maliciosos), para diferentes quantidades de pares honestos; os resultados são apresentados em termos relativos à relação entre pares maliciosos e honestos, no eixo x , e percentual de downloads falhos, no y . Nessa análise, o tempo de um download de um conteúdo varia com as características dos pares do enxame. Assim sendo, o tempo aceitável para “término” de um enxame (todos downloads são completados) foi determinado em função do tempo (denominado T_{fim}) em que último par completa o download em um cenário *sem* ataques. O tempo máximo adotado na Figura 7 é igual a $\alpha \times T_{fim}$, com $\alpha = 10$.

Analisando a figura, nota-se primeiro que os enxames são em geral afetados pelo ataque de Eclipse e as curvas apresentam comportamento consistente. Segundo, como demonstrado pela comparação das curvas entre si, o número absoluto de pares tem impacto na efetividade do ataque: as curvas que são proporcionalmente menos afetadas (menos falhas) são aquelas com menor número absoluto de pares honestos. Terceiro, nota-se que quando a proporção de maliciosos é inferior a 5:1, o efeito dos ataques é inexistente ou negligível. Por outro lado, a partir de 5:1, o efeito de ataques cresce agressivamente; quando é 6:1 em um enxame com 100 pares corretos, os 600 maliciosos são capazes de provocar quase 40% de falhas nos downloads. Por fim, a melhor relação entre custo benefício para um ataque parece se situar em torno de 20:1, pois faria com que pelo menos 85% dos downloads falhasse independentemente do tamanho do enxame. Generalizando, existe um número arbitrário e finito de pares maliciosos que é

suficiente para tornar um enxame BitTorrent inútil. Explorando a identidade fraca característica dessa arquitetura, é possível que um usuário mal intencionado automatize o processo de registro e assuma um número bastante alto de identidades virtuais.

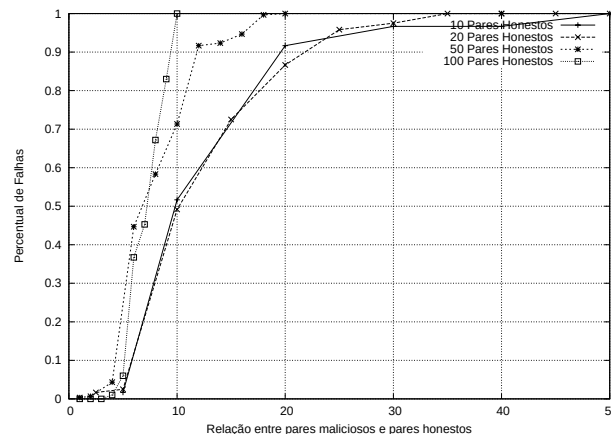


Figura 7. Impacto da quantidade de maliciosos no percentual de downloads falhos, para diferentes tamanhos de enxame.

7. Conclusão

Este trabalho trata de vulnerabilidades em BitTorrent, com ênfase em dois ataques baseados em Sybils (identidades virtuais falsas). O trabalho tem três contribuições. Primeiro, identificar os ataques e avaliar seu impacto potencial (trabalhos anteriores se concentram na avaliação do mecanismo de incentivo, enquanto este representa a primeira contribuição em termos de ataques ao BitTorrent). Segundo, aumentar o entendimento sobre o BitTorrent, documentando-se o comportamento de implementações populares. Terceiro, a criação de uma simulação discreta que mapeia o comportamento do protocolo, validada comparando-se com uma avaliação experimental em ambiente controlado.

Os resultados da avaliação demonstraram que BitTorrent é suscetível a ataques em que pares maliciosos mentem a posse de peças e tornam-nas mais raras, fazendo com que downloads em geral sejam atrasados em até 50%. Mostrou-se que esse ataque é efetivo mesmo quando é usado um número reduzido de maliciosos. Entretanto, mesmo em cenários extremos onde o número de mentirosos é igual ao número de pares corretos, não foram registrados casos em que um par não completa o download. Em contraste, tais falhas foram amplamente verificadas em ataques de Eclipse, em que o número de pares maliciosos suplantou largamente o número de honestos.

O artigo é uma primeira contribuição sobre o assunto vulnerabilidades de segurança em BitTorrent, e pode ser estendido de várias formas. Primeiro, existem outras vulnerabilidades na arquitetura, que não foram tratadas neste artigo por uma questão de foco. Segundo, o trabalho deve ser prosseguido através da proposta de mecanismos ou modificações no BitTorrent de maneira a eliminar ou reduzir o impacto de tais ataques. Terceiro, a avaliação de impacto foi, neste artigo, restrita a um conjunto de cenários simples, predominantemente homogêneos. Nesse contexto, vislumbra-se como continuidade ao trabalho a investigação de cenários mais ricos, particularmente com enxames de vida longa que se reciclam através de várias “gerações” de semeadores e sugadores.

Referências

Andrade, N., , Wagner, G., and Ripeanu, M. (2005). Influences on cooperation in bittorrent communities. In *ACM SIGCOMM P2P-ECON Workshop*.

- Barbera, M., Lombardo, A., Schembra, G., and Tribastone, M. (2005). A markov model of a freerider in a bittorrent P2P network. In *IEEE Global Telecommunications Conference (GLOBECOM '05)*, volume 2, pages 985–989, St. Louis, MO, USA.
- Bharambe, A. R., Herley, C., and Padmanabhan, V. N. (2005a). Some Observations on BitTorrent Performance. In *SIGMETRICS '05: Proceedings of the 2005 ACM SIGMETRICS international conference on Measurement and modeling of computer systems*, volume 33, pages 398–399, New York, NY, USA. ACM Press.
- Bharambe, A. R., Herley, C., and Padmanabhan, V. N. (2005b). Understanding and deconstructing BitTorrent performance. Technical Report MSR-TR-2005-03, Microsoft Research.
- BitTorrentProtocolSpecification (2006). Bittorrent Protocol Specification v1.0. <http://wiki.theory.org/BitTorrentSpecification>.
- Douceur, J. R. (2002). The Sybil Attack. In *1st International Workshop on Peer-to-Peer Systems*, pages 251–260.
- Guo, L., Chen, S., Xiao, Z., Tan, E., Ding, X., and Zhang, X. (2005). Measurements, analysis, and modeling of BitTorrent-like systems. In *IMC '05, 2005 Internet Measurement*, pages 35–48.
- Jun, S. and Ahamad, M. (2005). Incentives in BitTorrent induce free riding. In *ACM SIGCOMM Workshop on Economics of Peer-to-Peer systems (P2P-ECON)*, pages 116–121.
- Legout, A., Urvoy-Keller, G., and Michiardi, P. (2006). Rarest First and Choke Algorithms Are Enough. Technical report, INRIA.
- Liogkas, N., Nelson, R., Kohler, E., and Zhang, L. (2006). Exploiting bittorrent for fun (but not profit). In *5th International Workshop on Peer-to-Peer Systems (IPTPS'06)*.
- P2P-SeC (2006). P2P-SeC: Secure and Reliable Computing for Peer-to-Peer Networks. <http://p2p-sec.org>.
- Purandare, D. and Guha, R. (2006). Preferential and strata based p2p model: Selfishness to altruism and fairness. In *12th International Conference on Parallel and Distributed Systems, 2006. ICPADS 2006.*, volume 1, pages 561–570.
- Qiu, D. and Srikant, R. (2004). Modeling and performance analysis of BitTorrent-like peer-to-peer networks. *SIGCOMM Comput. Commun. Rev.*, 34(4):367–378.
- Sadok, D., Kamienski, C. K., Souto, E., Rocha, J., Domingues, M., and Callado, A. (2005). Colaboração na internet e a tecnologia peer-to-peer. In *XXV Congresso da Sociedade Brasileira de Computação (SBC 2005)*, pages 1407–1454.
- Simmcast (2006). Simmcast: an object-oriented simulation framework for protocol and network research. <http://inf.unisinos.br/Simmcast/>.
- Singh, A., Ngan, T.-W., Druschel, P., and Wallach, D. S. (2006). Eclipse attacks on overlay networks: Threats and defenses. In *25th Conference on Computer Communications (INFOCOM 2006)*. IEEE.
- Veciana, G. and Yant, X. (2003). Fairness, incentives and performance in peer-to-peer networks. In *Proceedings of the 41st Annual Allerton Conference on Communication, Control and Computing*.
- Yang, X. and de Veciana, G. (2004). Service capacity of peer to peer networks. In *INFOCOM 2004. Twenty-third Annual Joint Conference of the IEEE Computer and Communications Societies*, volume 4, pages 2242–2252.